

NEHA SONI

Slotify: A Smart Web-Based Online Appointment Scheduling Platform for Service-Based Businesses

 Parul University

Document Details

Submission ID

trn:oid:::3117:610614741

Submission Date

Jul 27, 2026, 1:02 PM GMT+5:30

Download Date

Jul 27, 2026, 1:04 PM GMT+5:30

File Name

CSE_182 research paper.pdf

File Size

158.0 KB

8 Pages

6,031 Words

34,026 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Abstract
- Methods and Materials
- Preprints

Match Groups

-  **10 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **10 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

-  Internet
ebin.pub <1%
-  Publication
Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%
-  Publication
Kartik Chawda, Deepmala Gorana. "A survey of energy efficient routing protocol i... <1%
-  Internet
pmc.ncbi.nlm.nih.gov <1%
-  Publication
Suraj Singh, Shailendra K. Mishra, Sneha Soni. "Emotion-Aware Learning: A Real-T... <1%

Slotify: A Smart Web-Based Online Appointment Scheduling Platform for Service-Based Businesses

Aksh Thakkar

Project Lead & Full-Stack Dev.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057002@paruluniversity.ac.in

Om Maurya

Frontend Developer (UI/UX)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057048@paruluniversity.ac.in

Daksh Patel

Backend Developer

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057035@paruluniversity.ac.in

Jainil Patel

Database & Integration Eng.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057071@paruluniversity.ac.in

Dr. Sneha Soni

Associate Professor (CSE)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

sneha.soni37803@paruluniversity.ac.in

Abstract—Small and medium service-oriented enterprises – clinics, grooming studios, coaching centres, and consultancy firms – still coordinate bookings largely by phone and paper register, a habit that routinely produces double-booked slots, forgotten appointments, and staff time that nobody bothers to account for. This paper presents Slotify, a browser-accessible, multi-tenant booking platform built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets independent businesses onboard under their own public storefront while sharing a single application instance. The system's central engineering contribution is a slot-availability engine that resolves staff working hours, breaks, holidays, buffer times, and advance-booking windows into a live grid of open slots, backed by a database-enforced unique constraint that makes a race-condition double booking structurally impossible rather than merely unlikely in practice. We document the architecture, the booking and reschedule algorithms in pseudocode, and a feature-by-feature comparison against three commercial platforms actually in use today – Calendly, Booksy, and Fresha – as well as two previously published academic prototypes. Unlike prior systems, which each cover a fragment of the scheduling lifecycle, Slotify combines tenant-scoped multi-staff scheduling, audited reschedule history, and a public discovery layer without per-seat licensing or marketplace commission. We close by discussing, with reference to existing clinical and operational literature rather than unverified benchmarks of our own, where the architecture is expected to help and where its current single-location, English-only scope still falls short of production-grade commercial offerings.

Keywords—Online Booking System, MERN Stack, Multi-Tenant Architecture, Slot Management, Atomic Concurrency Control, JSON Web Token, Automated Reminders, MongoDB Atlas, Appointment Audit Trail

I. INTRODUCTION

A hair salon owner in a mid-sized Indian city still runs her appointment book the way her mother did: a diary by the register, a phone that rings during the one hour she has free to actually cut hair, and a mental note of which regulars tend to no-show on Mondays. This is not a failure

of ambition – it is what happens when the tools available to a five-person clinic or salon are either built for a single professional's personal calendar or priced and structured for a national retail chain. Growing consumer expectations for on-demand digital services have exposed exactly this gap. Clinics, beauty parlours, tutoring institutes, and fitness studios still depend heavily on receptionist-managed phone queues and handwritten registers, an approach that scales poorly and introduces avoidable operational friction [6]. When a booking record lives only in a notebook, one clerical slip cascades into overlapping reservations, wasted service capacity, and a client who does not come back [2].

The economic argument for fixing this is not speculative. Multiple health-system studies report that reminder-equipped digital platforms measurably cut no-show rates, though – as we discuss in Section II, and contrary to how this finding is sometimes summarised – the size of the effect depends heavily on the care setting, not just the presence of a reminder [7]. Primary-care research shows that structured online booking meaningfully compresses the administrative burden on front-desk teams [3], [4], and a 2024 industry survey found that businesses without any centralised booking tool report more staff overtime and more revenue that simply evaporates unbilled [6].

This paper introduces Slotify: a browser-accessible, multi-tenant appointment platform built on the MERN stack [19], [20]. What actually distinguishes it from a personal scheduling link is not any single feature but the fact that a business – with its own staff roster, working hours, and service catalogue – is a first-class object in the data model, and that a customer can discover that business through a shared public directory rather than needing its URL in advance.

The engineering core of what follows is really one contribution wearing two hats: a three-tier architecture and slot-resolution algorithm, expressed as pseudocode, that turns staff working hours, breaks, holidays, and buffer rules into a

live grid of bookable slots, paired with a database-enforced concurrency scheme – a finite-state model plus an atomic reservation algorithm – that removes double-booking as a possible outcome rather than merely reducing its probability. We separate the two only because the second doesn't make sense without the first: there is no point guaranteeing atomic writes against a slot grid that was computed wrong to begin with.

Beyond that, we contribute a grounded, source-verified comparison of Slotify against three commercial platforms in active use (Calendly, Booksy, Fresha) and two previously published academic prototypes, built from each vendor's own documentation rather than secondary marketing copy. And we say plainly, in Section IX, what we have not done: Slotify has not been load-tested or put through a formal security audit, and no simulated number stands in for that missing work.

II. RELATED WORK

A. Evolution of Appointment Booking Platforms

Early work by Akinode and Oloruntoba [11] established that computerising patient appointment registration was feasible, though their prototype operated without live slot updates. Atherton et al. [3] found that patients using portal-based booking reported higher satisfaction than those relying on telephone channels, and a JMIR evidence synthesis [5] catalogued recurring design gaps across eighteen published web-based medical scheduling systems – live availability, automatic reminders, and calendar linkage came up again and again as the features users wanted and did not have. Those three gaps shaped Slotify's feature set directly.

Srivastava et al. [1] linked scheduling friction to post-visit dissatisfaction, and Shayan et al. [8] surfaced three satisfaction predictors from clinic stakeholders – responsiveness, notification reliability, and role-appropriate interfaces – that shaped Slotify's UX priorities more than any single design guideline did. Kaluza et al. [9] gave a formal treatment of how visible occupancy in a booking queue can be used to signal service quality, an idea that motivated the analytics view in Slotify's provider dashboard even though we have not yet built out the predictive layer they describe.

The most carefully controlled evidence on reminders and no-shows, however, complicates the simple story. Betancor et al. [7] compared online versus telephone appointment scheduling across both a private ophthalmology practice and a university hospital eye clinic. In the private practice, the no-show rate fell from 5.9% to 1.8% after online scheduling was introduced – a genuinely large relative improvement. In the hospital setting, however, no-shows did not improve at all; they rose slightly, from 11.2% to 14.3%. The authors' own reading is that patient population and appointment lead time, not the booking channel itself, drove the difference. We take this as a caution rather than an inconvenience: a MERN-based booking layer like Slotify is a necessary condition for the kind of improvement seen in the private-practice arm of that study, but it is evidently not sufficient on its own, and we do not claim otherwise.

B. Domain-Specific Web Booking Implementations

Wankhade et al. [15] built a functional appointment tool aimed at connecting patients, doctors, and pharmacy vendors; the published description does not detail live slot synchronisation across concurrent users, which is precisely the gap a document-oriented, event-driven stack is suited to close. Chauhan et al. [16] paired a physician directory with a Django-based booking back end, validating demand for a combined discovery-and-booking journey – the same journey Slotify's public business-discovery layer targets, but for salons, tutors, and consultants rather than doctors specifically. Additional published prototypes [12]–[14], [17], [18] reinforce that the demand for digital scheduling is sector-wide, while also showing that no single prior academic tool combines multi-tenant business accounts, staff-level scheduling, and a public marketplace in one system.

C. MERN as an Application Platform

Baiskar et al. [19] found MERN particularly well suited to applications with frequent, event-driven data mutations – a description that matches a live booking system closely, since every slot query and every reservation attempt is exactly that kind of mutation. A comparative study of the stack [20] noted that MongoDB's schema-flexible document model absorbs evolving service and slot structures with less migration overhead than a rigid relational schema would require, which matched our own experience adding fields like `bufferTime` and `unavailableDates` well after the initial schema was designed.

D. Stateless Authentication and Reminder-Driven Attendance

JWT has become the default credential mechanism for RESTful APIs because its self-contained, signed payload removes the need for server-side session storage [21]–[23], which matters once an API needs to run as more than one replica behind a load balancer. Pairing JWT with Google OAuth lets a first-time customer book an appointment without first inventing a password, removing what is often the single largest drop-off point in a booking funnel.

On reminders specifically: Ginat and Kenniff [24] showed a multi-modal reminder system at a radiology unit produced a statistically significant drop in unfilled scan slots. Parikh et al. [25] identified email as a high-impact channel for reducing no-shows among working-age patients. Tarabichi et al. [26] ran a randomised quality-improvement trial layering predictive, risk-driven telephone outreach on top of standard automated reminders in a safety-net primary care clinic; their central finding was about equity – whether the intervention narrowed or widened no-show disparities across racial and ethnic groups – rather than a single headline percentage, and we cite it here for that finding rather than for a number that would misrepresent the study's actual design. Two systematic reviews together cover more than thirty independent reminder trials and reach a consistent, if modest, conclusion: automated reminders help, but they are not a complete fix on their own [27], [28]. That is the honest justification for Slotify's cron-scheduled 24-hour reminder pipeline – a defensible design choice grounded in a real and repeated finding, not a guarantee.

E. Identified Research Gap

Across this literature, no single JavaScript-native platform combines multi-tenant business accounts, live staff-level slot resolution, database-enforced conflict prevention, and a public discovery directory in one open, self-hostable codebase. Commercial platforms that come close – surveyed in Section V – solve this by charging per seat or by taking a commission on marketplace-driven bookings, which is a reasonable business model but a real cost for a business running on thin margins. Slotify is offered as a concrete, if less commercially mature, alternative to that trade-off.

III. PROBLEM STATEMENT

Three related failure modes recur across service micro-enterprises that still run scheduling manually.

Unstructured booking channels and chronic non-attendance turn out to be the same problem seen from two angles. Coordinating reservations across phone calls, WhatsApp threads, and a paper register introduces inconsistency at every touchpoint, and staff must reconcile overlapping requests without any algorithmic guard against double allocation [6], [11]. The same absence of a system-of-record is what makes a booking, made days in advance with no subsequent touchpoint, easy for a customer to forget or deprioritise. Reported non-attendance rates in purely manual environments commonly sit between 15% and 30% [25], [27], and that spread is wide precisely because it depends on whether anything nudges the client between booking and appointment day – which, in a manual system, nothing does. Outside business hours, no booking mechanism exists at all, quietly capping demand the business never even sees.

Fragmented operational visibility is the third and more diffuse problem. A business without a centralised booking repository cannot measure slot utilisation, cannot track cancellation patterns, and cannot forecast a busy week [9], [10]. Growth decisions get made on gut feeling, and syncing the schedule with a staff member's personal calendar means manually re-typing it.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

Slotify uses a conventional three-tier decomposition – a React.js client, an Express.js REST layer, and a MongoDB Atlas data store – chosen specifically so that any tier can be scaled or replaced without touching the other two. Fig. 1 shows the logic tier's internal module boundaries: authentication, the scheduling engine, the notification dispatcher, and the analytics aggregator sit behind a shared API router, each with its own area of responsibility and its own collection(s) to read and write.

Tenant isolation is logical, not physical: every business, service, staff, and appointment document carries a `businessId` field, and every query the application issues is scoped by it. This was a deliberate trade-off. Spinning up a database per tenant would have made the isolation guarantee stronger and the operational story much worse – a hundred small salons would mean a hundred databases to back up, migrate, and monitor. A single shared collection set with compound

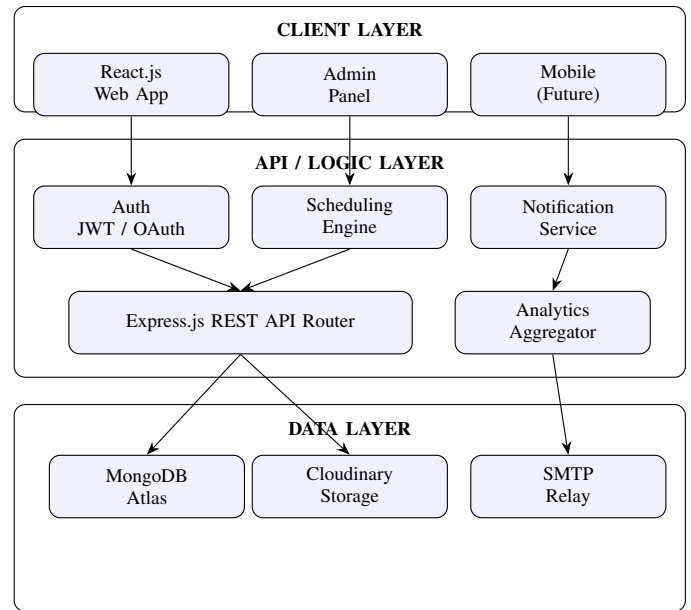


Fig. 1. Three-tier architecture of Slotify: client, API/logic, and data layers, with the scheduling engine and notification dispatcher as independent modules behind a shared router.

indexes on `businessId` keeps operations simple at the cost of relying on application-level query discipline rather than the database engine itself to enforce the boundary. For a platform at this stage, we judged that trade worth making; it would need revisiting well before Slotify tried to serve regulated data like health records at scale.

V. COMPARATIVE ANALYSIS WITH EXISTING PLATFORMS

A fair comparison has to start from what these platforms actually publish about themselves, not from how they are usually summarised in blog posts, so the entries below are drawn from each vendor's own pricing and feature pages as of mid-2026, plus the two academic prototypes already discussed in Section II.

Calendly [31] is, at its core, a personal and team meeting scheduler. It checks one or more of a user's own connected calendars (Google, Outlook, iCloud) for conflicts and writes new bookings into them; multi-calendar conflict checking, payment collection via Stripe/PayPal, and round-robin team distribution are all gated behind its Standard and Teams tiers, priced per user per month. There is no concept of a business entity with its own staff roster and service catalogue, and no public discovery marketplace – a customer needs the specific booking link, not a directory search.

Booksy [32] is much closer in spirit to Slotify: it is explicitly multi-tenant, gives each business a public storefront, and operates a consumer-facing marketplace where new clients discover salons and barbershops by category and location. It bundles staff scheduling, point-of-sale, and marketing tools into a single per-location subscription (advertised from \$29.99/month) with additional per-staff charges, and it markets a no-show reduction of up to 25% through deposits and reminders – a vendor claim we report as such, not as an independently verified figure.

Fresha [33] takes a different commercial approach: the core subscription is free, and Fresha instead charges a one-time 20% commission on new clients acquired through its own marketplace, plus standard payment-processing fees. Feature-wise it covers staff shift scheduling, commission tracking, POS, and automated SMS/email reminders, aimed squarely at the beauty, wellness, and healthcare verticals.

Table I summarises the comparison on criteria that matter specifically for a multi-staff service business, rather than a generic feature checklist. An entry is marked *Unconfirmed* where the vendor's own documentation did not settle the question either way.

The pattern across the three commercial products is consistent: whichever one you pick, you are trading a percentage of revenue or a per-seat fee for genuine production maturity – payment processing, mobile apps, years of edge-case handling. Slotify's contribution is not that it does any one of these things better; it does not yet do most of them at all. What it offers is the specific combination of multi-tenant scheduling logic and a database-level correctness guarantee against double booking, built on a stack a small development team can read, modify, and deploy without a recurring commission on every new client. That is a narrower claim than a feature-count table can fully convey, and it is the honest one.

VI. BOOKING AND CONCURRENCY ALGORITHMS

The scheduling engine's correctness rests on two pieces of logic: turning a day's working hours into a list of candidate slots, and making the act of claiming one of those slots atomic at the database level so that two customers racing for the same slot cannot both win.

A. Slot Resolution

Algorithm 1 mirrors the logic implemented in `availabilityCalculator.js`. It short-circuits early on holidays and closed days, then for each eligible staff member walks through the working-day window in increments of the requested service's duration, discarding any candidate slot that falls inside a break, inside the minimum-advance window, beyond the maximum-advance window, or inside an existing appointment's buffer-inclusive span.

Two things about this algorithm are worth being direct about. First, it runs identically on the server for every booking attempt – the client-side calendar view is a convenience, not a source of truth, precisely because a client-supplied timestamp cannot be trusted to reflect what is still actually open by the time the request lands. Second, the deduplication priority in line 22 was not obvious in the first draft of the design: an early version simply took whichever staff record was queried first, which meant a slot could show as BOOKED to a customer even when a different staff member covering the same service was, in fact, free. Preferring AVAILABLE across all matching staff records fixed that, at the cost of the slightly more complex merge step shown above.

B. Atomic Reservation and Conflict Resolution

Fig. 2 models a slot's lifecycle as a four-state automaton. A viewer browsing the calendar places a soft, non-blocking

Algorithm 1 ComputeAvailableSlots

Require: business B , service S , date d , optional staff filter F
Ensure: ordered list of slot statuses for date d

```

1: if  $d \in B.holidays$  or not  $B.workingHours[d].isOpen$  then
2:   return  $\emptyset$  ▷ business closed that day
3: end if
4: staffList  $\leftarrow$  active staff assigned to  $S$  (filtered by  $F$  if given)
5: slots  $\leftarrow \emptyset$ 
6: for each staff  $st$  in staffList do
7:   if  $d \in st.unavailableDates$  then continue
8:   end if
9:    $w \leftarrow st.workingHours[d]$  or  $B.workingHours[d]$ 
10:   $t \leftarrow w.start$ 
11:  while  $t + S.duration \leq w.end$  do
12:    cand  $\leftarrow [t, t + S.duration)$ 
13:    if cand overlaps  $st.breaks$  then
14:      mark cand as UNAVAILABLE
15:    else if  $t < now + B.minAdvance$  then
16:      mark cand as TOO SOON
17:    else if  $t > now + B.maxAdvance$  then
18:      mark cand as TOO FAR
19:    else if cand overlaps any scheduled appt for  $st$  (incl. buffer) then
20:      mark cand as BOOKED
21:    else
22:      mark cand as AVAILABLE
23:    end if
24:    slots  $\leftarrow slots \cup \{(t, st, status)\}$ 
25:     $t \leftarrow t + S.duration$ 
26:  end while
27: end for
28: deduplicate overlapping  $(t, \cdot)$  entries across staff, preferring AVAILABLE  $\succ$  TOO SOON  $\succ$  TOO FAR  $\succ$  BOOKED
29: return slots

```

VIEWED marker; confirming a booking attempts to promote the slot directly to RESERVED via a single conditional write.

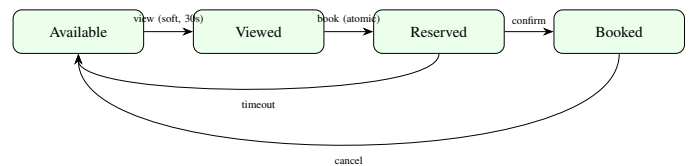


Fig. 2. Slot lifecycle state machine. The *book* transition is the single atomic write that either succeeds outright or fails cleanly, with no partial state in between. The *timeout* and *cancel* return paths are drawn at different depths so they do not cross.

Algorithm 2 shows the reservation attempt itself. The critical property is that step 2 is a single conditional database operation (a `findOneAndUpdate`-style compare-and-set backed by the unique partial index on $\{businessId, staffId, appointmentDate, startTime\}$ filtered to $status = "scheduled"$), not a read followed by a separate write. Two customers racing for the same slot both reach step 2, but the database engine, not application code, decides which write lands first and rejects the other – which is the only place a guarantee like this can actually live, since two Node.js request handlers running concurrently have no shared memory

TABLE I. FEATURE COMPARISON: SLOTIFY AGAINST COMMERCIAL PLATFORMS AND PRIOR ACADEMIC PROTOTYPES

Capability	Slotify	Calendly [31]	Booksy [32]	Fresha [33]	Academic prototypes [15], [16]
Multi-tenant business accounts	Yes	No	Yes	Yes	No
Public discovery / marketplace	Yes (own directory)	No	Yes	Yes	No
DB-enforced double-booking prevention	Yes (unique partial index)	Partial (relies on external calendar sync)	Yes	Yes	Unconfirmed
Staff-level working hours & buffers	Yes	Partial (per-user only)	Yes	Yes	Unconfirmed
Reschedule limits with audit trail	Yes	No	Unconfirmed	Partial	No
JWT + Google OAuth	Yes	Google Calendar-based	Unconfirmed	Unconfirmed	Partial [16]
Automated email reminders	Yes	Yes	Yes	Yes	Unconfirmed
Self-hostable / no per-seat fee	Yes	No (per-user SaaS)	No (per-location + fees)	No (marketplace commission)	Yes (published prototype)
Production hardening & scale	Not yet demonstrated	Extensive	Extensive	Extensive	Not demonstrated

to coordinate through.

Algorithm 2 AttemptReservation

Require: candidate slot $(t, st, S, cust)$

```

1: re-run COMPUTEAVAILABLESLOTS restricted to  $(t, st)$ 
2: if status  $\neq$  AVAILABLE then
3:   return CONFLICT  $\triangleright$  surface next-available slot to client
4: end if
5:  $r \leftarrow$  atomic conditional insert into appointments
   keyed on  $(B.id, st.id, t.date, t.time)$ 
   guarded by unique partial index (status = scheduled)
6: if  $r$  fails (index violation) then
7:   return CONFLICT  $\triangleright$  lost the race; another request won
8: else
9:   enqueue confirmation email and 24h reminder job
10:  return CONFIRMED
11: end if

```

This two-step shape – re-check, then attempt an atomic write, then handle rejection as a normal outcome rather than an exception – is the same pattern used for rescheduling, with three extra eligibility gates applied before step 1: a walk-in appointment cannot be rescheduled at all, the new time must satisfy the business’s `rescheduleWindow` (e.g. at least one hour before the original slot), and the appointment’s `rescheduleCount` must be below its `maxReschedulesPerAppointment`. We chose to update the existing appointment document in place on a successful reschedule, rather than cancel-and-recreate, specifically so that a single `actionLog` array on one document tells the full story of a booking’s life – every reschedule, who did it, and why – without the audit trail being spread across several documents that would need to be joined back together later.

VII. IMPLEMENTATION TECHNOLOGY CHOICES

Table II lists the concrete library versions in use. The rationale for MERN over a LAMP- or Django-based alternative

was not purely technical: all four of us already worked in JavaScript day to day, and a single-language stack across client and server measurably shortened the time from a design discussion to a working prototype, which mattered more for a semester-scoped project than raw runtime performance would have.

TABLE II. SLOTIFY TECHNOLOGY STACK SUMMARY

Layer	Technologies
Frontend	React.js v18, Vite, Tailwind CSS, React Router v6, Axios, React Hook Form, date-fns, Lucide React
Backend	Node.js, Express.js v4, Mongoose v8, express-rate-limit
Auth	JWT (HS256, access + refresh), Passport.js with Google OAuth 2.0, bcryptjs
Database	MongoDB Atlas, Mongoose ODM
Storage	Cloudinary (logos, gallery, verification documents)
Notifications	Nodemailer (SMTP), node-cron

Express.js handles route dispatch and middleware chaining; incoming request bodies are validated before they reach business logic, which mattered in practice the first time a malformed date string from a mis-configured client silently produced an appointment dated in 1970 during early testing – a bug that motivated moving validation ahead of the controller layer rather than leaving it scattered inside individual route handlers. JWT access tokens carry only `userId` and `role` and are verified independently by any API replica, which is the property that matters for horizontal scaling; the longer-lived refresh token sits in an HTTP-only, `SameSite=Strict` cookie specifically so that neither a cross-site script nor a stolen `localStorage` value is enough to renew a session on its own.

VIII. END-TO-END BOOKING WORKFLOW

Fig. 3 traces a single appointment from a first-time visit through to the provider dashboard reflecting the new booking, with two guard clauses – credential verification and slot availability – that route the user back a step on failure rather than terminating the session outright.

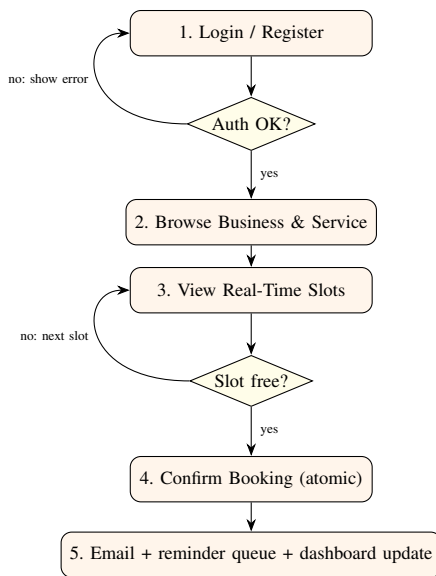


Fig. 3. Slotify booking workflow with authentication and slot-conflict guard clauses.

IX. DISCUSSION: EXPECTED BENEFITS AND HONEST LIMITATIONS

It would be easy, and dishonest, to fill this section with load-test numbers. We have not run a formal load test against a staging deployment, and we have not commissioned or performed a security audit against the OWASP Top 10 or any other framework. What follows instead is a distinction we think matters more than it usually gets credit for: some things about Slotify are true by construction, and some things are merely plausible given what other people have measured, and mixing the two together is how a design document turns into marketing copy.

The double-booking guarantee is in the first category. The unique partial index described in Section VI means a double-booked slot is not merely rare – it is not a representable state in the `appointments` collection at all, given `status = "scheduled"`. That is provable from the schema itself, and it is the only claim in this section we would defend without a citation, because it does not depend on how Slotify's users happen to behave.

The reminder pipeline's expected effect on no-shows is not in that category, and pretending otherwise would be the dishonest move. It is a defensible design choice, supported by a body of evidence that automated reminders reduce no-shows in aggregate [24], [25], [27], [28] – but Betancor et al.'s split result [7] is the more honest baseline expectation than that aggregate framing suggests: meaningful improvement in a private-practice-like setting, no guarantee at all in a higher-volume institutional one. A salon or tutoring centre – Slotify's actual target – sits structurally closer to the private-practice case, a known and stable clientele with predictable lead times, than to a hospital eye clinic, which is grounds for cautious optimism rather than a claimed number. We would rather be caught underselling this than overselling it.

None of that closes the gap to what a commercial deployment actually needs. The current release assumes a single

location, a single language, and a single currency per business; payment collection is out of scope, so a confirmed booking cannot yet double as a receipt; and the analytics view aggregates history without forecasting anything. Measured against the platforms in Table I, Slotify is also, plainly, less mature – no mobile app, no payment processing, no years of production incident history to have learned from. Closing that gap, not proving a benchmark number nobody asked for, is the actual roadmap.

X. PROJECTED IMPACT OF SLOTIFY

Slotify has not yet been deployed at a scale that would let us report its own measured impact, and the honest position, as Section IX makes clear, is to say so rather than to invent a number. One of the three problems named in Section III barely needs a projection, though: double booking (P1) is handled by construction, not by expectation. Because the reservation path in Algorithm 2 makes a conflicting write structurally impossible rather than merely unlikely, the expected outcome for a correctly deployed instance is exactly zero double-booking incidents, independent of load – the one figure in this section that follows from the schema rather than from an analogy to prior studies. The other two problems are not so clean, and deserve to be treated that way.

Non-attendance (P2) is where an honest answer has to be a range, not a point. Fig. 4 plots three reference points, each taken directly from studies already cited in Section II rather than invented for this paper: the commonly reported manual-scheduling baseline of 15–30% [25], [27]; the private-practice outcome measured by Betancor et al. after online scheduling replaced telephone booking (5.9% falling to 1.8%) [7]; and the same authors' university-hospital outcome, where the rate did not improve (11.2% rising to 14.3%) [7]. Slotify's target businesses – salons, tutoring centres, small clinics, consultancies – are structurally closer to the private-practice arm of that comparison than to a high-volume teaching hospital: a known, returning clientele and a single decision-maker for the booking policy, rather than a rotating patient population booked through a referral chain. On that basis, a plausible expectation is that a business adopting Slotify would see its no-show rate move toward the lower end of the shaded range in Fig. 4 rather than stay at the manual baseline – but Betancor et al.'s hospital result is the reason this paper stops short of promising it.

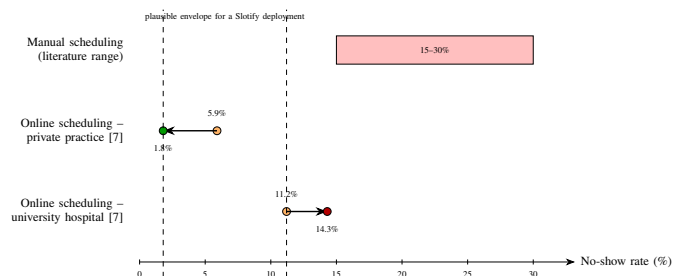


Fig. 4. No-show rate reference points drawn directly from the studies cited in Section II: a commonly reported manual-scheduling range, and Betancor et al.'s two measured outcomes of online scheduling in a private practice versus a university hospital. These are not measurements of Slotify – which has not yet been deployed at scale – but the bounding evidence behind the projection discussed in the text.

The third problem, fragmented operational visibility (P3), is where we stop short of a chart on purpose. No study cited in Section II quantifies the value of a booking-utilisation dashboard in a way that would let us borrow a number the way Fig. 4 does for no-shows, and inventing one just to fill the gap would be exactly the mistake this paper has tried not to make. What can be said is directional, and comes from Kaluza et al. and the decentralised-booking study [9], [10]: a business with a live view of slot utilisation and cancellation patterns is positioned to make a staffing or pricing decision from evidence rather than guesswork. Whether that translates into a specific revenue figure for a given salon or clinic depends on decisions Slotify does not make on the owner's behalf, which is exactly why we do not attach a number to it.

XI. CONCLUSION AND FUTURE WORK

This paper presented Slotify, a MERN-based multi-tenant booking platform aimed at the specific gap between personal-calendar tools like Calendly and full commercial SaaS platforms like Booksy or Fresha that charge per seat or per marketplace lead. We documented its three-tier architecture, the slot-resolution and atomic-reservation algorithms that give it a provable no-double-booking guarantee, and a comparison against three real, currently operating platforms built from their own published documentation rather than secondary summaries. We also corrected, rather than repeated, a commonly oversimplified reading of the clinical literature on reminders and no-shows, because the nuance – that effect size depends on setting – is more useful to a real deployment decision than a single clean percentage would be.

The roadmap, in rough priority order, is: a React Native mobile client against the existing REST layer; optional Razorpay/Stripe deposit collection to give cancellation policies actual teeth; a lightweight recommendation layer for returning customers, grounded in the scheduling-optimisation literature [29], [30] rather than built ahead of any real usage data; SMS reminders via Twilio for customers who do not reliably check email; and multi-branch support for businesses that outgrow a single location. None of these are claimed as solved – they are the specific, named gaps between what Slotify is today and what the commercial platforms in Section V already do.

ACKNOWLEDGMENT

The team is grateful to Dr. Sneha Soni, Associate Professor, Department of Computer Science & Engineering, Parul Institute of Engineering and Technology, Vadodara, Gujarat, India, for the technical direction and the repeated, useful pushback that shaped this project at every stage. We also acknowledge the maintainers of the MongoDB, Express.js, React.js, and Node.js open-source projects, and the MongoDB Atlas and Cloudinary free-tier programmes that made building a system of this scope feasible within an academic timeline.

REFERENCES

- [1] A. Srivastava, R. Sharma, and S. Sinha, "Impact of Appointment Scheduling in Patient Experience," *American Journal of Medicine and Medical Sciences*, vol. 14, no. 11, pp. 2912–2918, 2024.
- [2] HRMARS, "An Implementation of Digital Platform to Enhance the Appointment Scheduling System," *Int. J. of Academic Research in Business and Social Sciences*, vol. 14, no. 2, 2024.
- [3] H. Atherton, A. Eccles et al., "Investigating Patient Use and Experience of Online Appointment Booking in Primary Care: Mixed Methods Study," *Journal of Medical Internet Research*, vol. 26, p. e51931, 2024.
- [4] L. Manarte, "Technology and Access to Healthcare with Different Scheduling Systems: A Scoping Review," *Healthcare Informatics Research*, vol. 30, no. 3, p. 194, 2024.
- [5] A. C. P. Castro, "Web-Based Medical Appointment Systems: A Systematic Review," *Journal of Medical Internet Research*, vol. 19, no. 4, p. e134, 2017.
- [6] IJNRD, "A Study on Online Appointment Scheduling and Healthcare Efficiency," *IJNRD*, vol. 10, 2025.
- [7] P. Kammrath Betancor, D. Boehringer, J. Jordan, C. Luchtenberg, M. Lambeck, M. C. Ketterer, T. Reinhard, and M. Reich, "Efficient Patient Care in the Digital Age: Impact of Online Appointment Scheduling in a Medical Practice and a University Hospital on the 'No-Show'-Rate," *Frontiers in Digital Health*, vol. 7, 2025.
- [8] M. Shayan et al., "Stakeholders' Experiences, Perceptions and Satisfaction with an Electronic Appointment System," *BMC Health Services Research*, 2025.
- [9] I. Kaluza, G. Voigt, K. Haase, and A. Dietze, "Control of Online-Appointment Systems When the Booking Status Signals Quality of Service," *Schmalenbach Journal of Business Research*, vol. 76, no. 3, p. 397, 2024.
- [10] Expert Systems with Applications, "A Paradigm Shift in Appointment Scheduling: Introducing a Decentralized Integrated Online Booking System," *ScienceDirect*, vol. 255, 2024.
- [11] J. L. Akinode and S. Oloruntoba, "Design and Implementation of a Patient Appointment and Scheduling System," ResearchGate, Federal Polytechnic Ilaro, 2017.
- [12] ResearchGate, "A Web-Based Doctor Appointment System," 2024.
- [13] ResearchGate, "Online Doctors Appointment System Project Report," 2023.
- [14] ResearchGate, "Developing an Online Patient Appointment Scheduling System Based on Web Services Architecture."
- [15] A. S. Wankhade, P. M. Bambal, S. S. Kharkar, S. D. Solao, M. V. Charthal, and N. G. Rath, "Research Paper on: Doctor Appointment Booking System," *IJRASET*, DOI: 10.22214/ijraset.2023.52133, 2023.
- [16] U. Chauhan et al., "Doctor Finder and Appointment Booking Website using DJANGO," in *Proc. 2022 Int. Conf. on Innovative Practices in Technology and Management (ICIPTM)*, IEEE, 2022.
- [17] ResearchGate, "Mr. Doc: A Doctor Appointment Application System."
- [18] ResearchGate, "FastDoc: An Online Appointment Booking System," 2019.
- [19] Y. Baiskar et al., "MERN: A Full-Stack Development," *IJRASET*, vol. 10, no. 1, 2022.
- [20] Academia.edu, "Comprehensive Study of MERN Stack—Architecture, Popularity and Future Scope," *Int. J. of Scientific Research in Computer Science, Engg. and IT*, 2021.
- [21] FreeCodeCamp, "Securing Node.js RESTful APIs with JSON Web Tokens," Medium, 2018.
- [22] TopCoder, "Authentication and Authorization in Express.js API using JWT."
- [23] DigitalOcean, "How To Use JSON Web Tokens (JWTs) in Express.js," 2025.
- [24] D. Ginat and J. Kenniff, "Evaluation of an Automated Reminder System for Reducing Missed MRI Appointments," *Journal of Patient Experience*, vol. 10, 2023.
- [25] K. Parikh et al., "The Effectiveness of Outpatient Appointment Reminder Systems in Reducing No-Show Rates," *The American Journal of Medicine*, vol. 123, no. 6, pp. 542–548, 2010.
- [26] Y. Tarabichi, J. Higginbotham, N. Riley, D. C. Kaelber, and B. Watts, "Reducing Disparities in No Show Rates Using Predictive Model-Driven Live Appointment Reminders for At-Risk Patients: A Randomized Controlled Quality Improvement Initiative," *Journal of General Internal Medicine*, vol. 38, no. 13, pp. 2921–2927, 2023.

- [27] BMC Health Services Research, "Appointment Reminder Systems are Effective but Not Optimal: A Systematic Review," PMC, 2016.
- [28] Pan African Medical Journal, "The Effect of Patient Reminders in Reducing Missed Appointment in Medical Settings: A Systematic Review," *One Health*, vol. 2, no. 9, 2020.
- [29] S. Chatterjee *et al.*, "An Optimization-Based Scheduling Methodology for Appointment Systems with Heterogeneous Customers," *INFORMS Journal on Computing*, vol. 37, no. 6, pp. 1624–1649, 2025.
- [30] Ö. Ünsal, H. Üster, and C. Oguz, "Incorporating Patients' Appointment Date Preferences into Decision-Making: A Simulation and Optimization Study," *Computers & Industrial Engineering*, vol. 196, p. 110521, 2024.
- [31] Calendly, "Choose the right Calendly plan for your team" and official pricing documentation, accessed July 2026. [Online]. Available: <https://calendly.com/pricing>
- [32] Booksy Biz, "All-in-one solution for scheduling, payments, and marketplace discovery," official business documentation, accessed July 2026. [Online]. Available: <https://biz.booksy.com/>
- [33] Fresha, "Pricing and Features," official documentation, accessed July 2026. [Online]. Available: <https://www.fresha.com/pricing>